

MODULE 4

MULTIPROCESSOR SYSTEM

- Consists of several processors that share the same physical memory.

MOTIVATIONS FOR MULTIPROCESSOR SYSTEM

- 1.Enhanced performance
- 2.Fault tolerance

BASIC MULTIPROCESSOR SYSTEM ARCHITECTURE

- Based on whether a memory can be directly accessed by a processor or not, there are two types of multiprocessor systems
 1. Tightly coupled vs loosely coupled system
 2. UMA vs NUMA vs NORMA Architecture

Tightly Coupled vs Loosely Coupled System

- All the processors share the same memory-tightly coupled

Eg:) Multimax of Encore Corporation, Flex/32 of Flexible Corporation and FX of Sequent Computers

- Not only is the main memory is partitioned and attached to processors, but each processors has its own address space-loosely coupled

Eg) Intel's Hypercube

A memory contention occurs when two or more processors simultaneously try to access the same memory module

UMA vs NUMA vs NORMA Architecture

- **UMA(UNIFORM MEMORY ACCESS)**

Main memory is located at a central location such that it is equidistant from all the processors in terms of access time

- All the processors have same access time to main memory
- Processors may also have private memories to cache data for better performance

Eg

Multimax of Encore Corporation, Balance of Sequent and VAX 8800 of Digital Equipment

UMA vs NUMA vs NORMA Architecture

- NUMA ARCHITECTURE

Main Memory is partitioned and the partitions are attached to the processors.

A processor can directly access the memory attached to any other processor, but the time to access the memory attached to other processors is much higher than the time to access its own memory partition.

Example of Numa Architecture are Cm* of CMU and Butterfly machine of BBN Laboratories.

UMA vs NUMA vs NORMA Architecture

- NORMA ARCHITECTURE

Main Memory is partitioned and the partitions are attached to the processors.

A processor cannot directly access the memory attached to any other processor

Messages are sent over the interconnection to exchange information.

Eg: Intel's Hypercube

INTERCONNECTION NETWORKS FOR MULTIPROCESSOR SYSTEMS

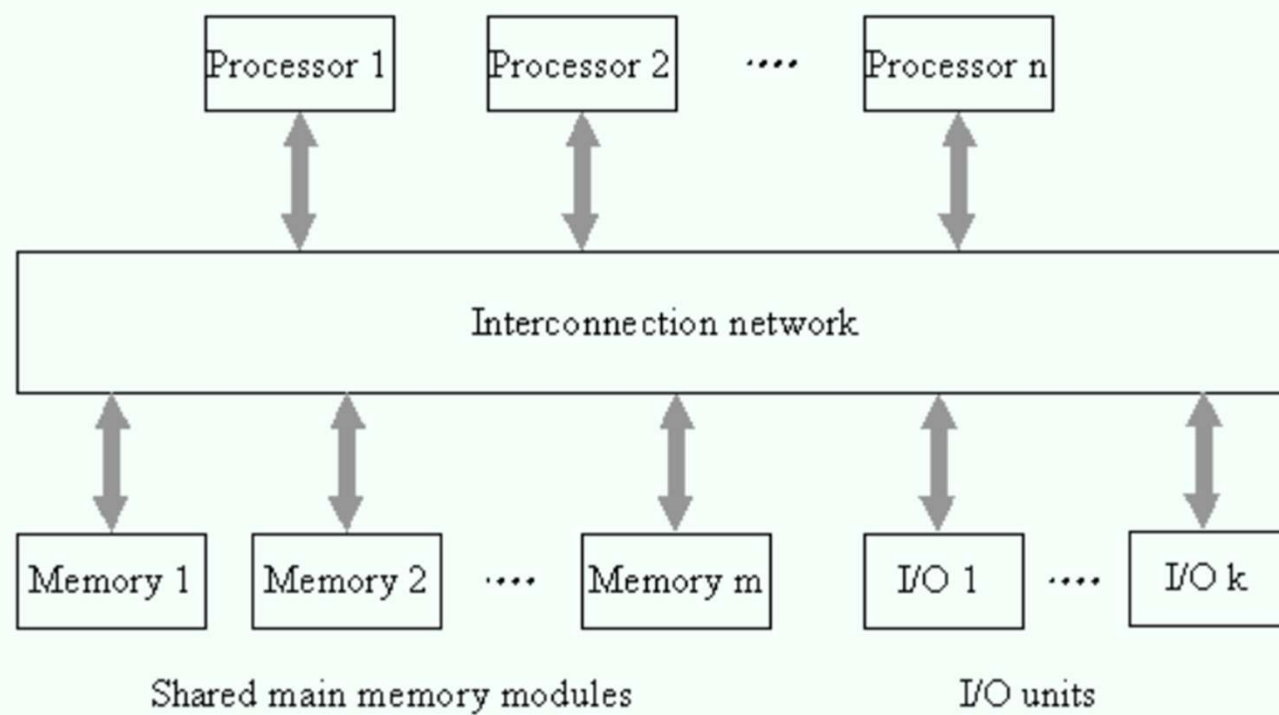
- Provides data transfer facility between processors and memory modules for memory access
- The design of the interconnection network is the most crucial hardware issue in the design of multiprocessor systems
- Generally circuit switching is used to establish a connection between processors and memory modules.

Various type of interconnection networks include

- Bus
- Cross –bar switch
- Multistage Interconnection Network

1. Bus

- single bus is considered the simplest way to connect multiprocessor systems.
- In its general form, such a system consists of N processors, each having its own cache, connected by a shared bus.
- However aside from shared memory the bus is also a source of contention because the bus can support only one processor memory communication at a time Eg for bus based Multi processor system-Encore Corporation's Multimax
- The typical size of such a system varies between 2 and 50 processors. The actual size is determined by the traffic per processor and the bus bandwidth. Hence there is a limit. So in order to overcome this we use multiple bus



2.Cross bar Switch

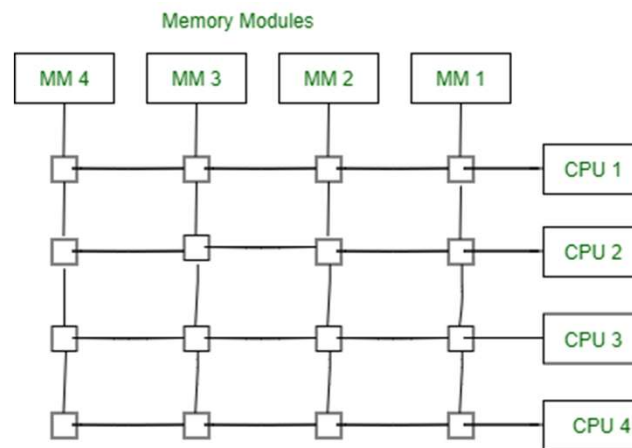


Figure - Crossbar Switch System

- It's a matrix which has a cross point
- Provides exclusive connection between any processor memory pair.
- Thus all n processors can concurrently access memory modules provided that each processor is accessing a different memory module. ($n \leq m$)
- A cross bar does not face contention at the interconnection network level.
- A contention can occur only at the memory module level.

3. MultiStage Interconnection Network

- It's a compromise between a bus and a cross bar.
- Permits simultaneous connections between several processor memory pairs and more cost-effective than a cross bar.
- A typical multistage interconnection network consists of several stages of switches.
- The output of the switches in a stage is connected to the inputs of the switches in the next stage.
- The routing path between a processor and a memory module pair is given by a binary string that is derived from the binary address of the processor and memory module.

In an $N \times N$ multistage interconnection n/w requires only $(N/2) \times \log N$ switches as compared to N^2 switches in an $N \times N$ crossbar

Design issues in multiprocessor systems

- Cache coherency
- Snooping
- False Sharing
- Processor affinity
- Programming Models

Design issues in multiprocessor systems

1.Cache coherency

If thread one in the first processor is operating on the same data as thread one in the second processor, each processor will have its own copy of the data in its cache.

The system must ensure that in case of changes done by any processor must be reflected accurately to whole copies by the threads. This is called maintaining **cache coherency**.

These operations are performed by the system hardware.

So, at a given point in time, the value of a data item may be different between a processor's cache and System Memory. The system maintains cache coherency by detecting when a data item is about to be accessed and performing the operations needed to update the data item with the correct value before allowing access to complete.

Design issues in multiprocessor systems

2.Snooping

Snooping is used to keep the track of cache subsystem and each of its cache lines.

This technique monitors all transactions that take place on the system bus and detects when a read or write operation takes place on an address that is in its cache.

When the cache subsystem “snoops” a read on the system bus to an address in its cache, it changes the state of the cache line to “shared”. If it snoops a write to that address, it will change the state of the cache line to “invalid”.

- Because it is snooping the system bus, the cache subsystem will know if it has the only copy of a data item in its cache.
- If such a data item is updated by its own CPU, the cache subsystem will change the state of the cache line from “exclusive” to “modified”. If the cache subsystem snoops access to that data item by another processor, it can stop that access from occurring, update the data item in System Memory, and then allow the other processor’s access to proceed.
- It will also change the state of the cache line with the data item to “shared

3.False Sharing

The selection of a programming model looks at cases where the designer knows that data is shared between threads. False sharing occurs when separate data items that are accessed by separate threads are allocated to the same cache line.

Since data access causes an entire cache line to be read into the cache from System memory, if one data item in the cache line is shared, all of the data items in that cache line will be treated as shared by the cache subsystem.

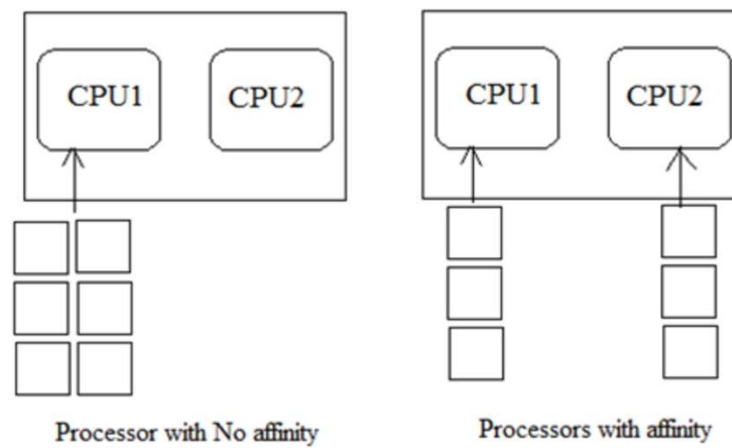
Two data items could be updated in unrelated transactions by two threads running on different cores but, if the two items are in the same cache line, the cache subsystem will have to update the System Memory in order to maintain cache coherency setting up a condition where pin ponging can occur.

4. Processor affinity

Processor affinity also called as CPU pinning. It allows allocating or de-allocating of threads or processes to the same CPU.

By using this technique, the CPU can manage its tasks in a given time. This happens when either CPU has multiple processors or multiple cores.

In the figure below, processes of interrupts are mapped to a single CPU, it means no affinity at all. But on another side of the diagram, two CPU's are sharing the processes to manage the workload. This is the case of processor affinity. In this way, a process or interrupt will be assigned to only designated CPU or core.



5.Programming Models

There are two programming models in software designing, functional decomposition, and data (domain) decomposition.

These help in assigning the work to threads.

The task of functional decomposition is to divide the software work into small units or threads. One thread holds the multiple operations but each thread has given a particular function to perform fast. In data decomposition, data sets are decomposed into parts and software works on it independently. Each thread operates on separate data component parallel.

1.THE SEPARATE SUPERVISOR CONFIGURATION

In separate supervisor configuration,all processors have their own copy of the kernel,supervisor and data structures.

There are some common data structures for the interaction among processors,the access to which it is protected by the synchronization mechanism.

- Each. processor has its own I/O devices and file system.
- Very little coupling among processors and each processor acts as an autonomous independent system
- It is difficult to perform parallel execution at a single task.
- This configuration is inefficient because the supervisor/kernel/data structure code is replicated for each processor.

2.THE MASTER-SLAVE CONFIGUARTION

- In the master slave configuration,one processor called the master,monitors the status and assigns works to all the processors and the slaves.
- Slaves are treated as a schedulable pool of resources by the master.Such an OS is simple because it runs only on the master processor.
- Since the OS is executed by a single processor it is effiecient and its implementation is easy.
- The master slave configuration permits the parallel execution of a single task,where a task can be broken into several subtasks and the subtasks can be scheduled on multiple processors concurrently.

3.THE SYMMETRIC CONFIGURATION

In a Symmetrical multiprocessing system, each processor executes the same copy of the operating system, takes its own decisions, and cooperates with other processes to smooth the entire functioning of the system. The [CPU](#) scheduling policies are very simple. Any new job submitted by a user can be assigned to any processor that is least burdened. It also results in a system in which all processors are equally burdened at any time.

The symmetric multiprocessing [operating system](#) is also known as a "shared every-thing" system, because the processors share memory and the Input output bus or data path. In this system processors do not usually exceed more than 16.

However the concurrent access to the shared data structures of the supervisor needs to be controlled inorder to maintain their integrity.

The simplest way to achieve this is to treat the entire operating system as a critical section and allow only one processor to execute the operating system at one time.

This method is called the floating master method because it can be viewed as a master slave configuration where the master “floats” from one processor to another.

THREADS

- User-level Threads
- Kernal-level Threads
- First-Class Threads.
- Scheduler Activation

1.User level Threads

In user level threads,a run time library package provides the routines necessary for thread management operations.

These routines are linked at runtime to applications Kernal intervention is not required for the management of threads

2. Kernel level Threads

- Kernel-level threads are handled by the operating system directly and the thread management is done by the kernel. The context information for the process as well as the process threads is all managed by the kernel. Because of this, kernel-level threads are slower than user-level threads.
- **Advantages of Kernel-Level Threads**
- Some of the advantages of kernel-level threads are as follows –
- Multiple threads of the same process can be scheduled on different processors in kernel-level threads.
- The kernel routines can also be multithreaded.
- If a kernel-level thread is blocked, another thread of the same process can be scheduled by the kernel.
- **Disadvantages of Kernel-Level Threads**
- Some of the disadvantages of kernel-level threads are as follows –
- A mode switch to kernel mode is required to transfer control from one thread to another in a process.
- Kernel-level threads are slower to create as well as manage as compared to user-level threads.

3.First class Threads

- Was developed as part of the Psyche parallel OS.
- Kernal processors are used to implement the virtual processor that execute user level threads.
- Creating many virtual processors in the same address space and assigning them to different physical processors provide parallelism.
- In Psyche,a thread package creates and maintains the state of the threads in user space.
- Most of the thread operations,such as creation,destruction,synchronization and the context switching of threads are handled by the thread package

Coarse-grained parallelism

- In coarse-grained parallelism, a program is split into large tasks. Due to this, a large amount of computation takes place in processors. This might result in load imbalance, wherein certain tasks process the bulk of the data while others might be idle. Further, coarse-grained parallelism fails to exploit the parallelism in the program as most of the computation is performed sequentially on a processor. The advantage of this type of parallelism is low communication and synchronization overhead.
- [Message-passing](#) architecture takes a long time to communicate data among processes which makes it suitable for coarse-grained parallelism

- However coarse grain resource allocation and protection is in the domain of the kernel.
- Under first class threads to overcome the problems associated with the user level threads,three mechanism are provided to communicate between the kernel and the user package.
- These communication occur without any kernel traps

Description of these mechanisms follows

- 1.The kernel and the thread package share important data structures.The kernel managed data is made available to the thread package through read-only access
- 2.The kernel provides the thread package with software interrupts(signals,upcalls) whenever a scheduling decision is required.
- 3.Scheduler interfaces are provided to enable the sharing of data abstractions between dissimilar thread packages.

4.Scheduler Activations

Used to overcome the disadvantages of user-level threads has been developed .

Under this scheme,communication between the kernel and a user level thread package is structured in terms of scheduler ACTIVATIONS.

A scheduler activation has three roles.

- 1.It serves as an execution context for running user level threads.
- 2.It notifies the user level thread system of kernel events
- 3.It provides space in the kernel for saving the processor context of the activation's current user level thread when the thread is stopped by the kernal

Notifying Kernel level Events to the User level thread system

- To notify the thread system of kernel level events, the kernel creates a new scheduler activation assigns it to a processor and then upcalls into the user space.
- When the user level thread blocks in the kernel space, the kernel creates a new scheduler activation to inform the thread system that the thread has blocked.

Notifying User level Events to the Kernal level thread system

- The thread system notifies the kernel whenever the thread system enters a state wherein it has more processors than runnable threads or has more runnable threads than the number of assigned processors.

PROCESS SYNCHRONIZATION

- Issues in processor Synchronization
- The Test and Set Instruction
- The Swap Instructions
- The Fetch and ADD Instruction of the UltraComputer
- SLIC Chip of the Sequent
- Implementation of Process Wait
- The Compare and Swap Instruction

1.Issues In Processor Synchronization

- Busy-waiting, busy-looping or spinning is **a technique in which a process repeatedly checks to see if a condition is true**, such as whether keyboard input or a lock is available.
- Busy waiting by processors can cause excessive traffic to the interconnection network,thereby degrading system performance.
- It may also causes bandwidth consumption.
- To overcome this problem,multiprocessor systems provide instructions to atomically read and write a single memory location(in main memory)

- However if access to a shared data constitutes several instructions, then primitives such as lock and unlock are needed to ensure mutual exclusion

2.The Test and Set Instruction

The Test and Set instruction atomically reads and modifies the contents of a memory location in one memory cycle.It is defined as follows.

Function Test-and-Set(var m:Boolean):Boolean;

Begin

Test-and-Set:=m;

m:=true

end;

- The test and set instructions returns the current value of variable m and sets it to true. This instruction can be used to implement P and V operations on a binary Semaphore S , in the following way (S is implemented as a memory location)

$P(S)$: while Test-and Set(S)

do nothing

$V(S)$: $S := \text{false}$;

Initially, S is set to false. When a $P(S)$ operation is executed for the first time, Test and Set(S) returns a false value (and sets S to true) and the while loop of the $P(S)$ operation terminates. All subsequent executions of $P(S)$ keep looping because S is true until a $V(S)$ operation is executed

2.The Swap Instruction

- The swap instruction atomically exchanges the contents of two variables(eg,memory locations).It is defined as follows(x and y are two variables)

```
Procedure swap(var x,y:Boolean);
```

```
Var temp:Boolean;
```

```
begin
```

```
    temp:=x;
```

```
    x=y;
```

```
    y=temp;
```

```
end;
```

P and V operations can be implemented using the swap instruction in the following way
(P is a variable private to the processor and S is a memory location);

```
P(S): p:=true;  
Repeat swap(S,p) until p=false;  
V(S):S=false;
```

Clearly, the above two implementations of the P operation employ busy-waiting and therefore increase the traffic on the interconnection network.

Another problem with the test and set instructions is that if n processors execute any of these instructions in the same memory location, the main memory will perform n such operations on the location even though only one of these operations succeed.

4.The Fetch and Add Instruction of the UltraComputer

- It's a multiple operation memory access instruction that atomically adds a constant to a memory location and returns the previous content of the memory location.
- The instruction is defined as follows
- Function Fetch-and-Add(m:integer,c:integer);

```
Var temp:integer;
```

```
Begin
```

```
Temp:=m;
```

```
m=m+c;
```

```
return temp;  
end;
```

The fetch and add instruction is powerful and it allows the implementation of P and V operations on a general semaphore S, in the following manner.

```
P(S):while (Fetch-and-Add(S,-1)<0) do  
Begin  
Fetch-and-Add(S,1)  
While (S<=0) do nothing;  
end;  
V(S):Fetch-and-Add(S,1)
```

5.SLIC chip of the Sequent

SLIC stands for subscriber line interface card

The Sequence Balance/21000 multiprocessor system supports a low level mutual exclusion in hardware using a technique that is totally different from the previously discussed techniques, which use atomic multi-operation machine language instructions.

The main component of a Balance/21000 is a SLIC chip that supports many other functions in addition to low level mutual exclusion

- A SLIC chip contains 64 registers and supports the operations necessary for process synchronization.
- Each processor has a SLIC chip and all the SLIC chips are connected by a separate SLIC bus.
- Each bit in the SLIC chip is called a gate act as a separate lock and stores the status of the corresponding lock

- To lock a gate in the SLIC chip, a processor executes a lock-gate instruction.
- A lock-gate instruction is executed in the following manner.
- If the local copy indicates that the gate is closed, the instruction fails.
- Otherwise, the local SLIC of the processor attempts to close the gate by sending messages to SLIC chips over the SLIC bus

The following code implements P and V operations on a semaphore S;

```
P(S):while (lock-gate(S)=failed) do nothing;
```

```
V(S):unlock-gate(S)
```

6.Implementation of Process Wait

- Busy Waiting
- Sleep-Lock
- Queuing

7.The Compare and Swap Instruction

- The compare-and-swap instruction of IBM 370 is used in the optimistic synchronization of concurrent updates to a memory location.
- The instruction is defined as follows
- Compare-and-Swap(var r1,r2,m:integer);

```
var temp:integer;
```

```
Begin
```

```
temp:=m;
```

```
If temp=r1 then {m:=r2;z:=1}
```

```
else {r1:=temp;z:=0}
```

```
End;
```

The compare and swap instruction can be used to synchronize concurrent access to a shared variable, say m in the following manner.

```
r1:=m
```

```
label:r2:=r1+x
```

```
Compare-and swap(r1,r2,m)
```

```
If z=0 then go to label
```

Processor Scheduling

- Issues in Processor Scheduling
- Co-Scheduling of the Medusa OS
- Smart Scheduling
- Scheduling in the NYU Ultra computer
- Affinity Based Scheduling
- Scheduling in Mach OS

1.Issues In Processor Scheduling

- 3 major causes
- Preemption inside Spinlock controlled Critical Sections
- Cache Corruption
- Context Switching Overheads

2.Co-Scheduling of the Medusa OS

- In co –scheduling all the runnable tasks of an application needs to be preempted,all the tasks of that applications are preempted.
- Effectively co scheduling does context switching between applications rather than between tasks of several different applications.
- That is,all the tasks in an application are run for a timeslice,then all the tasks in another application are run for a timeslice and so on.
- Co scheduling alleviates the problem of tasks wasting resources in lock-spinning while they wait for a preempted task to release the critical section.However,it does not alleviate the overhead due to the context switching nor performance degradation due to cache corruption

3.Smart Scheduling

- The smart scheduler has two nice features.
- First it avoids preempting a task when the task is inside its critical section.
- Second it avoids the rescheduling of tasks that were busy waiting at the time of their preemption until the task that is executing the corresponding critical section releases it. When a task enters a critical section, it sets a flag. On the exit from the critical section, a task resets the flag.
- The smart scheduler eliminated the resource wastage due to a processor spinning a lock that is held by a task preempted inside its critical section. However it does not make any attempt to reduce the overhead due to the context switching nor to reduce the performance degradation due to cache corruption

4.Scheduling in the NYU Ultra Computer

- Combines the strategies of the previous two scheduling techniques.In this technique,tasks can be formed into groups and the tasks in a group can be scheduled in any of the following ways.
- A task can be scheduled or preempted in the normal manner.
- All the tasks in a group are scheduled or preempted simultaneously
- Tasks in a group are never preempted.

- In addition a task can prevent its preemption irrespective of the scheduling policy of its group. This provision can be used to effectively implement a spin lock
- Flexible

5.Affinity Based Scheduling

- Is the first scheduling policy to address the problem of cache corruption.
- In this policy,a task is scheduled on the processor where it is last executed.
- This policy alleviates the problem of cache corruption because it is likely that a significant portion of the working set of that task is present in the cache of that processor when the task is rescheduled.
- A task can save a significant amount of time that is normally spent in reloading its working set in the cache of the processor.
- Affinity based scheduling also decreases bus traffic due to the cache reloading.

- Affinity based scheduling, however restricts load balancing among processors because a task cannot be scheduled on any processor.
- Since tasks are always executed on processors for which they have an affinity, the system suffers from load imbalance because a task may wait at a busy processor while other processors are idle.

6.Scheduling in the Mach OS

- In the Mach OS an application or a task consists of several threads.
- A thread is the smallest independent unit of execution and scheduling in Mach.
- In the Mach OS all the processors of a multiprocessor are grouped in disjoint sets, called processors sets.
- These processors uses priority scheduling to execute the threads assigned to their processor set.
- When the processor is non empty, it selects the highest priority threads for execution

- If the local ready queue of the processor is non empty, it selects the highest priority threads for execution.
- Otherwise it selects the highest priority thread from the global ready queue for execution.
- If both the queues are empty, the processor executes a special idle thread until a thread becomes ready.
- When a thread runs out of timeslice of a processor it is preempted only if an equal or higher priority ready thread is present.
- Otherwise thread receives another timeslice at the processor

Hints In Mach OS

- Discouragement Hints-allows the scheduler to delay execution of the task.
- Can be mild, strong or absolute
- Handsoff hints-indicates that a specific thread should be run instead of the currently executing thread.
- Priority inversion problem

Memory Management: The Mach OS

- Design Issues
- The Mach Kernel
- Task Address Space
- Memory Protection
- Machine Independence
- Memory Sharing
- Efficiency Considerations

1.Design Issues

- Portability
- Data Sharing
- Protection
- Efficiency

2.The Mach Kernel

- Which provides only the basic primitives necessary for building parallel and distributed applications
- Adaptable and portable to a wide array of architectures

The Mach kernel supports five abstractions:
threads, tasks, ports, messages and memory objects

- Tasks and Threads
- Messages and Ports
- Memory Objects

3.Task Address Space

- Each task is assigned a single paged address space
- Size is limited due to h/w
- Mach treats address page as a sequence of bits.
- A page in a task address space is either allocated or unallocated.
- Regions
- The Mach virtual memory system supports such large, sparse address space
- Provides primitives for advance operations

4.Memory Protection

- Protection is enforced at page level
- Each allocated page has the following two protection codes associated with it viz current protection code and the maximum protection code

5. Machine Independence

- Splits the implementation into two parts viz machine independent and machine dependent based on the assumption of MMU's presence
- No assumption is made about the type of data structure.
- The machine independent part is responsible for maintaining high level machine independent data structures
- pmap module and pmap structures(machine dependent)
- A pmap structure corresponds to a page table
- The pmap module provides an interface to the machine independent part that are used by the machine independent part to notify the machine dependent part of any changes

Two types of independence to higher layers

- operating system independence
- paging store independence

6.Memory Sharing

- The Mach Virtual system allows sharing of memory via the inheritance mechanism
- The inheritance attribute of a page can take three values:

None,copy and share.

If the page is in the none inheritance mode,the child task does not inherit that page.

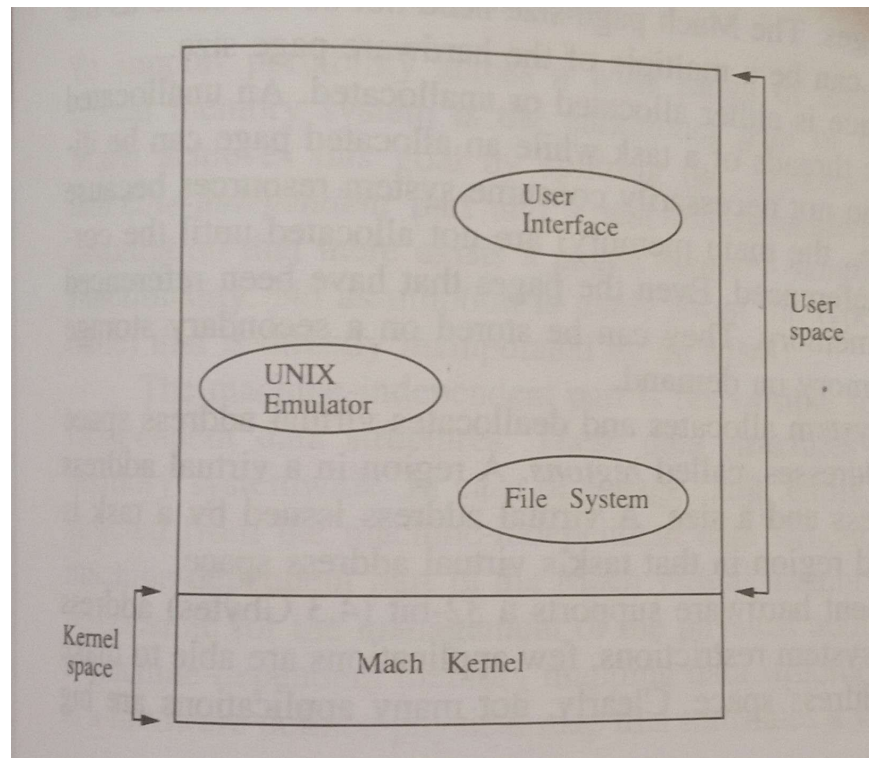
Copy mode-the child receives a copy of the page and the subsequent modifications to that page only affects the task making the modifications

7.Efficiency Considerations

- Parallel implementation
- Simplicity
- Lazy Evaluation

The copy on write operations

- Is an ex of lazy evaluation in Mach that optimizes memory space and CPU cycles
- Reduces memory overhead



8.Implementation:Data Structures and Algorithms

- Data Structures-has four basic data structures memory obejcts,pmap structures, resident page tables and address maps.
- Resident Page tables
- Address Maps

1. Resident Page Tables

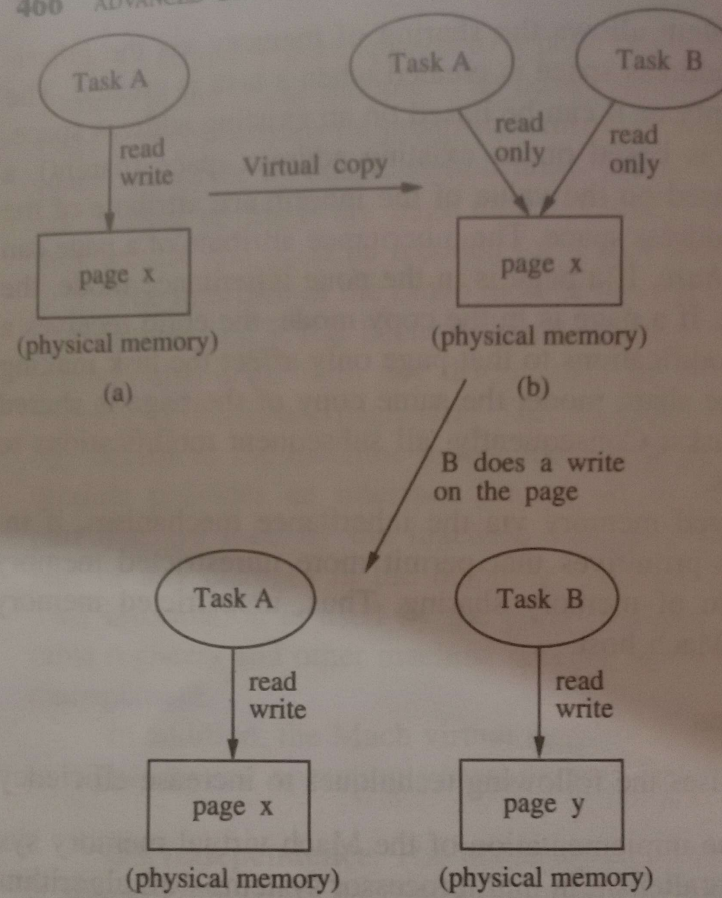
The Mach system treats physical memory as a cache for virtual memory objects

A page entry in the page table may be linked into the following lists.

- a. Memory object list
- b. Memory allocation queue
- c. Object/offset hash bucket

2.Address Maps

- Is a datastructure that maps contiguous chunks of virtual addresses in the address space of a task to memory objects.
- An address map is a doubly linked list of address map entries
- Efficient implementation of the most frequently performed operations on the address task,namely,page fault look ups,copy protection operations on memory region, and the allocation and deallocation of memory regions and efficient maintenance of sparse address space



Algorithms

- The Page Replacement Algorithms-Replacement algorithms in Mach is a modified FIFO algorithm that keeps all the physical memory pages in one of the following three FIFO queues.
- The free list
- The active list
- The inactive list

Page out daemon performs page replacement and management of these list

- The Page Fault Handler

The page fault handler is involved when a page is referenced for which there is either an invalid mapping or a protection violation. The page fault handler has the following responsibilities

(1) Validity and protection: It determines if the faulting thread has the desired access to the address by performing a look up in its task's address space

(2) Page look up: It attempts to find an entry for a cached page in its virtual to physical hash table. If the page is not represented, the kernel requests the data from the pager

(3)Hardware Validation: It informs the hardware physical map of the new virtual to physical mapping.

Locking Protocols

- a.Map locks
- b.Object locks
- c.Hash table bucket locks
- d.Busy page locks

9.Sharing of the memory

Sharing the same copy of a memory object by several tasks as long as all the tasks only read the memory object

Shadow objects

Sharing map.

Virtualization

- As in cloud technology, virtualization plays an important role to make things easy and efficiently done, virtualization also need to be done at the OS level also.
- It is also called OS-level virtualization is a type of virtualization technology which work on OS layer. Here the kernel of an OS allows more than one isolated user-space instances to exist.
- Such instances are called containers/software containers or virtualization engines. In other words, OS kernel will run a single operating system & provide that operating system's functionality to replicate on each of the isolated partitions.

- Such instances, called **containers** ([LXC](#), [Solaris containers](#), [Docker](#)), **Zones** ([Solaris containers](#)), **virtual private servers** ([OpenVZ](#)), **partitions**, **virtual environments** (VEs), **virtual kernels** ([DragonFly BSD](#)), or **jails** ([FreeBSD jail](#) or [chroot jail](#)),^[1] may look like real computers from the point of view of programs running in them.
- A computer program running on an ordinary operating system can see all resources (connected devices, files and folders, [network shares](#), CPU power, quantifiable hardware capabilities) of that computer. However, programs running inside of a container can only see the container's contents and devices assigned to the container.

The five levels of implementing Virtualization

- Virtualization is not that easy to implement. A computer runs an OS that is configured to that particular hardware. Running a different OS on the same hardware is not exactly feasible.
- To tackle this, there exists a hypervisor. What hypervisor does is, it acts as a bridge between virtual OS and hardware to enable its smooth functioning of the instance.
- There are five levels of virtualizations available that are most commonly used in the industry. These are as follows

1. Instruction Set Architecture Level (ISA)

- In ISA, virtualization works through an ISA emulation. This is helpful to run heaps of legacy code which was originally written for different hardware configurations.
- These codes can be run on the virtual machine through an ISA.
- A binary code that might need additional layers to run can now run on an x86 machine or with some tweaking, even on x64 machines. ISA helps make this a hardware-agnostic virtual machine.
- The basic emulation, though, requires an interpreter. This interpreter interprets the source code and converts it to a hardware readable format for processing.

2. Hardware Abstraction Level (HAL)

- As the name suggests, this level helps perform virtualization at the hardware level. It uses a bare hypervisor for its functioning.
- This level helps form the virtual machine and manages the hardware through virtualization.
- It enables virtualization of each hardware component such as I/O devices, processors, memory, etc.
- This way multiple users can use the same hardware with numerous instances of virtualization at the same time.
- IBM had first implemented this on the IBM VM/370 back in 1960. It is more usable for cloud-based infrastructure.
- Thus, it is no surprise that currently, Xen hypervisors are using HAL to run Linux and other OS on x86 based machines.

3.Operating System Level

- At the operating system level, the virtualization model creates an abstract layer between the applications and the OS.
- It is like an isolated container on the physical server and operating system that utilizes hardware and software. Each of these containers functions like servers.
- When the number of users is high, and no one is willing to share hardware, this level of virtualization comes in handy.
- Here, every user gets their own virtual environment with dedicated virtual hardware resources. This way, no conflicts arise.

4. Library Level

- OS system calls are lengthy and cumbersome. Which is why applications opt for APIs from user-level libraries.
- Most of the APIs provided by systems are rather well documented. Hence, library level virtualization is preferred in such scenarios.
- Library interfacing virtualization is made possible by API hooks. These API hooks control the communication link from the system to the applications.
- Some tools available today, such as vCUDA and WINE, have successfully demonstrated this technique.

5.Application level

- Application-level virtualization comes handy when you wish to virtualize only an application. It does not virtualize an entire platform or environment.
- On an operating system, applications work as one process. Hence it is also known as process-level virtualization.
- It is generally useful when running virtual machines with high-level languages. Here, the application sits on top of the virtualization layer, which is above the application program.
- The application program is, in turn, residing in the operating system.
- Programs written in high-level languages and compiled for an application-level virtual machine can run fluently here.

Five Levels of Virtualization

Application Level

JVM / .NET CLR

Library Level

WINE / vCUDA

Operating System Level

Virtual Environment / FVM

Hardware Abstraction Level

VMWare / Virtual PC

Instruction Set Architecture Level

BIRD / Dynamo

Hypervisors

Hypervisor

A hypervisor, also known as a virtual machine monitor or VMM, is **software that creates and runs virtual machines** (VMs). A hypervisor allows one host computer to support multiple guest VMs by virtually sharing its resources, such as memory and processing.

TYPE-1 Hypervisor:

The hypervisor runs directly on the underlying host system. It is also known as “Native Hypervisor” or “Bare metal hypervisor”. It does not require any base server operating system. It has direct access to hardware resources. Examples of Type 1 hypervisors include VMware ESXi, Citrix XenServer and Microsoft Hyper-V hypervisor.

- **Pros & Cons of Type-1 Hypervisor:**
- **Pros:** Such kind of hypervisors are very efficient because they have direct access to the physical hardware resources (like CPU, Memory, Network, Physical storage). This causes the empowerment of the security because there is nothing any kind of the third party resource so that an attacker couldn't compromise with anything.

- **Cons:** One problem with Type-1 hypervisor is that they usually need a dedicated separate machine to perform its operation and to instruct different VMs and control the host hardware resources.

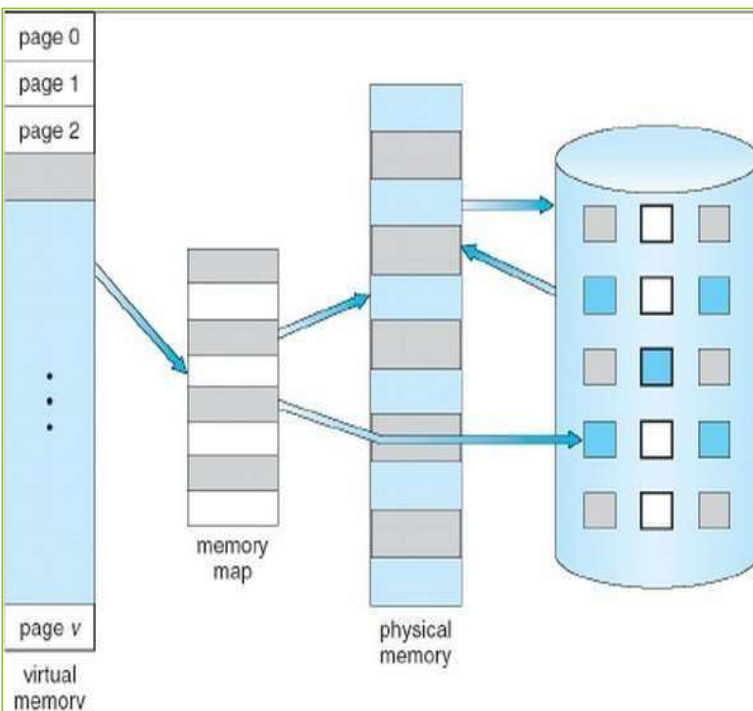
TYPE-2 Hypervisor:

- A Host operating system runs on the underlying host system. It is also known as 'Hosted Hypervisor'. Such kind of hypervisors doesn't run directly over the underlying hardware rather they run as an application in a Host system(physical machine). Basically, software installed on an operating system. Hypervisor asks the operating system to make hardware calls. Example of Type 2 hypervisor includes VMware Player or Parallels Desktop. Hosted hypervisors are often found on endpoints like PCs. The type-2 hypervisor is are very useful for engineers, security analyst(for checking malware, or malicious source code and newly developed applications).

- **Pros & Cons of Type-2 Hypervisor:**
- **Pros:** Such kind of hypervisors allows quick and easy access to a guest Operating System alongside the host machine running. These hypervisors usually come with additional useful features for guest machine. Such tools enhance the coordination between the host machine and guest machine.
- **Cons:** Here there is no direct access to the physical hardware resources so the efficiency of these hypervisors lags in performance as compared to the type-1 hypervisors, and potential security risks are also there an attacker can compromise the security weakness if there is access to the host operating system so he can also access the guest operating system.

MEMORY VIRTUALIZATION

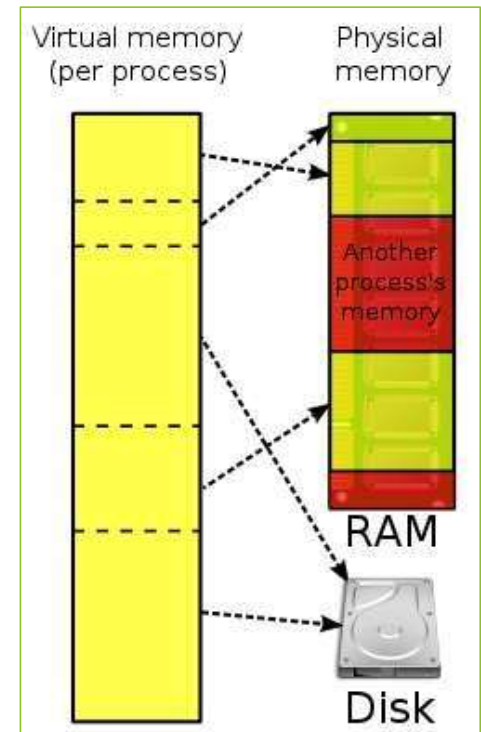
Virtual Memory



- Memory virtualization is seen as virtual memory, or swap, on servers and workstations.
- It enhances performance by providing greater memory capacity, without the expense of adding main memory.
- Instead, a portion of a disk drive serves as an extension of main memory.
- A *physical address* is the actual memory address of physical memory.
- Page faults occur when a logical address requires that a page be brought in from disk.

Virtual Memory

- Main memory and virtual memory are divided into equal sized pages.
- The entire address space required by a process need not be in memory at once. Some parts can be on disk, while others are in main memory.
- Further, the pages allocated to a process do not need to be stored contiguously-- either on disk or in memory.
- In this way, only the needed pages are in memory at any time, the unnecessary pages are in slower disk storage.



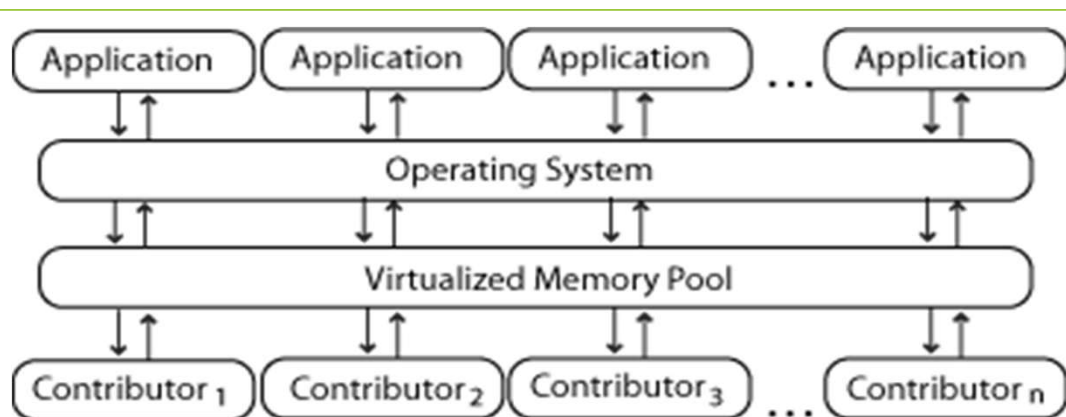
Importance of Virtual Memory

- When the computer runs out of physical memory it writes what it needs to remember to the hard disk in a swap file as virtual memory.
- If a computer running windows requires more memory or RAM, then it is installed in the system to run a program , it uses a small section of hard drive for this purpose.

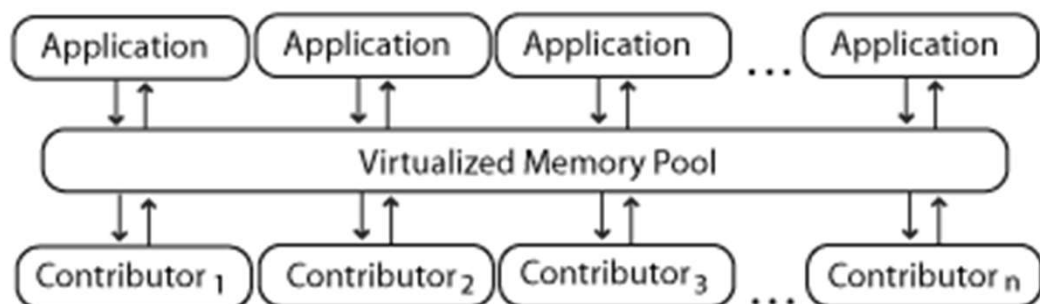


Example

- A 16 MB program can run in 4 MB space by carefully choosing which 4 MB to keep in memory at each instance, with pieces of the program being swapped between disk and memory as needed.



Memory Virtualization: Addressable Memory Implementation



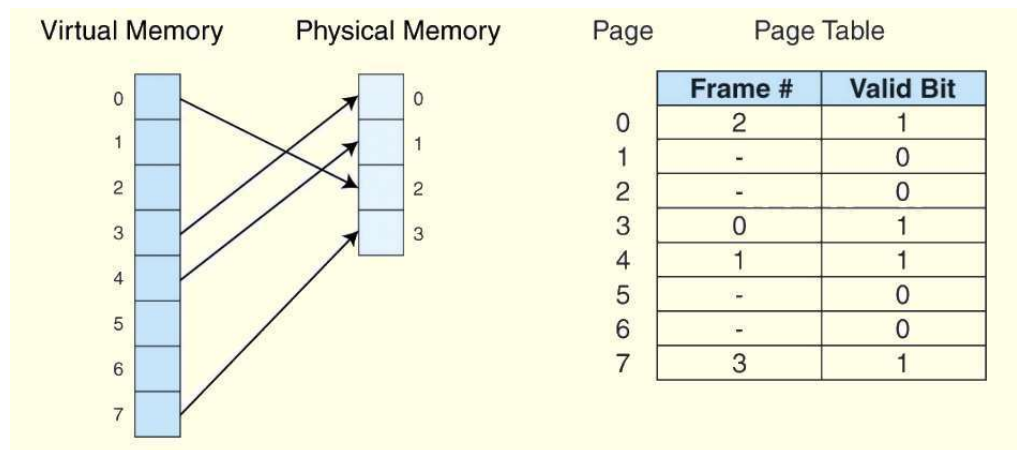
Memory Virtualization: Application Level Implementation

Demand Paged Virtual Memory

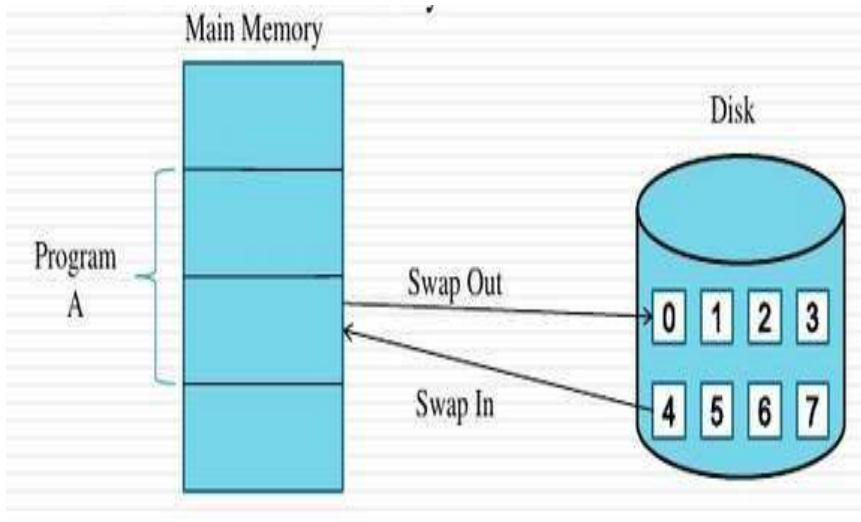
- The fundamental approach in implementing virtual memory is paging.
- To facilitate copying virtual memory into real memory, the operating system divides virtual memory into *pages*, each of which contains a fixed number of addresses.
- To accomplish this, the virtual address is divided into two fields: A *page* field, and an *offset* field.
- The page field determines the page location of the address, and the offset indicates the location of the address within the page.

Demand Paged Virtual Memory

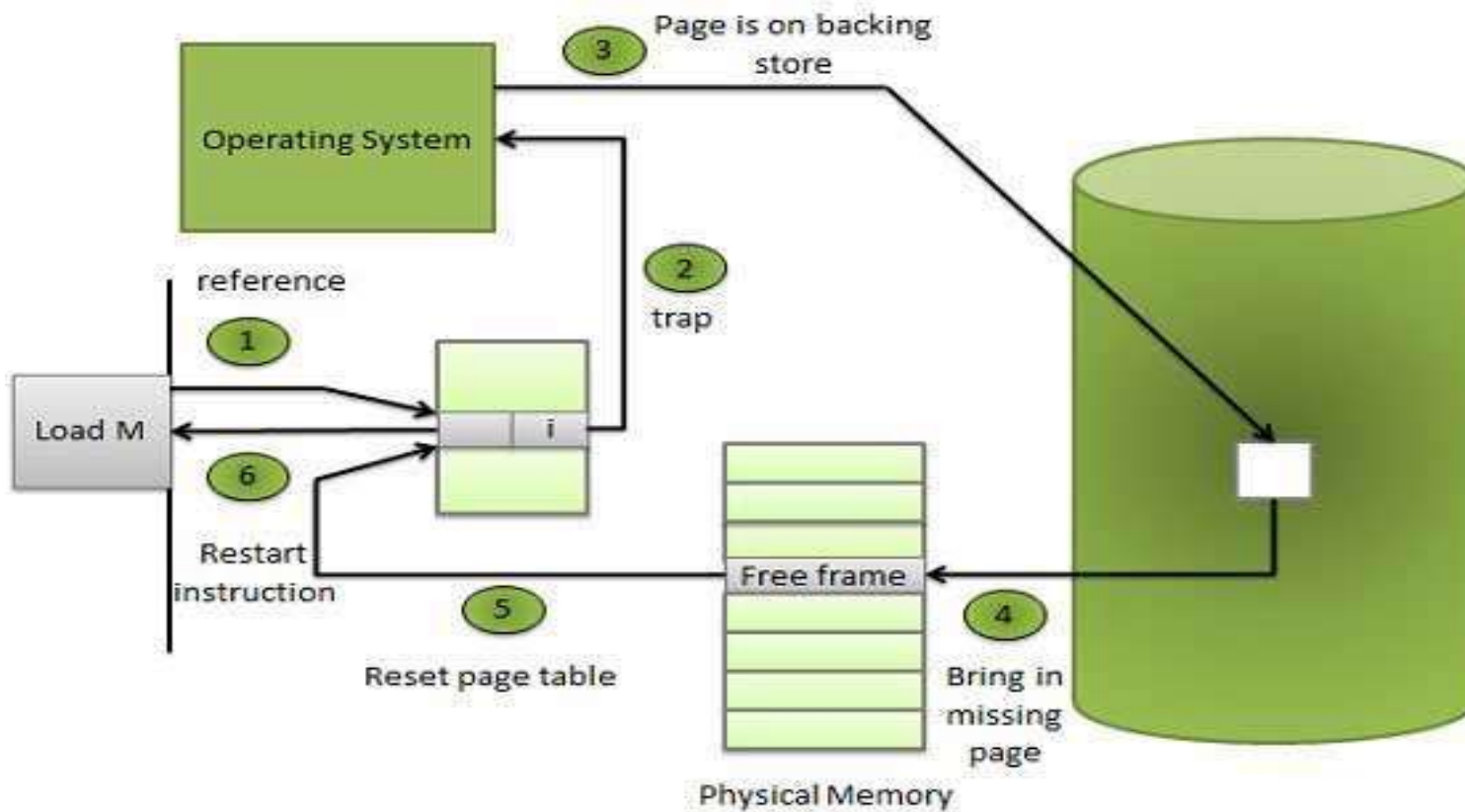
- The logical page number is translated into a physical page frame through a lookup in the page table
- Information concerning the location of each page, whether on disk or in memory, is maintained in a data structure called a *page table* (shown below).
- There is one page table for each active process.



Demand Paged Virtual Memory

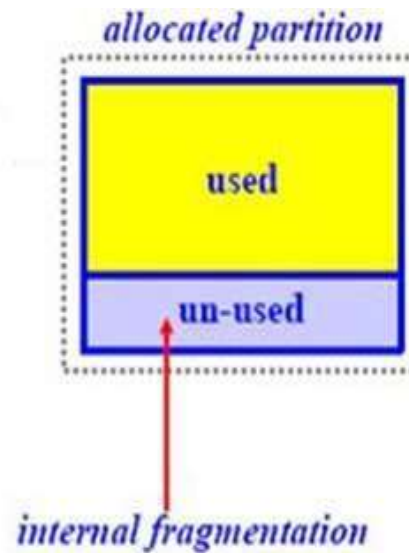


- When the pages are needed to execute a particular program, they are loaded.
- Pages that are never accessed are thus never loaded into the memory.
- This technique is known as Demand paging.



Internal Fragmentation

- Paging is subject to *internal* fragmentation because a process may not need the entire range of addresses contained within the page. Thus, there may be many pages containing unused fragments of memory.



Demand Segmented Virtual Memory

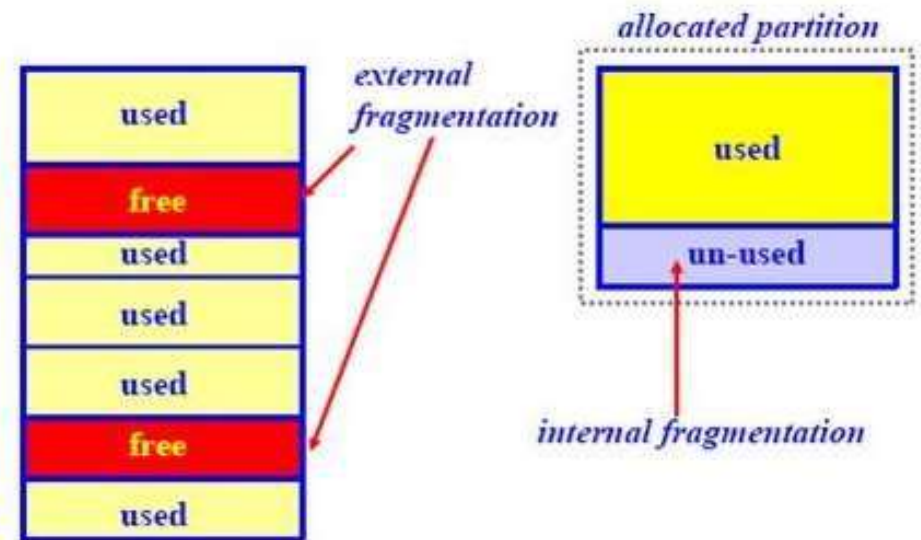
- Memory is divided into variable length segments.
- Each segment has a base physical address and length.
- Used when insufficient hardware is there to implement demand paging.
- Segment descriptor contains a valid bit to indicate whether the segment is currently in memory.
- If segment is in main memory, access continues,
- If not in memory, segment fault occurs.

Demand Segmented Virtual Memory

- A segment is located through its entry in a segment table, which contains the segment's memory location and a bounds limit that indicates its size.
- After a page fault, the operating system searches for a location in memory large enough to hold the segment that is retrieved from disk.
- Each segment has a page table. This means that a memory address will have three fields, one for the segment, another for the page, and a third for the offset.

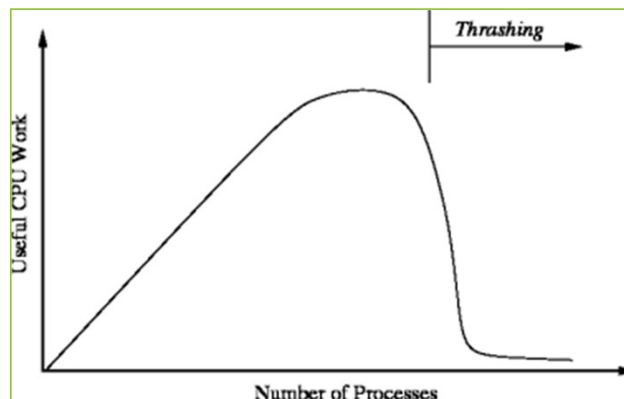
External Fragmentation

- Both paging and segmentation can cause fragmentation.
- Segmentation is subject to *external* fragmentation, which occurs when contiguous chunks of memory become broken up as segments are allocated and deallocated over time.



Thrashing

- When the hard drive is being overworked by moving information between the system memory and virtual memory excessively, thrashing is caused.
- When the system doesn't have enough memory, the system swap file is not properly configured, or too much is running on the computer and it has low system resources.
- Thrashing is bad on a hard drive because of the amount of work the hard drive has to do and if left unfixed will likely cause an early failure of the hard drive.



Advantages

- More applications can run at once.
- Larger applications can run with less real RAM without need to buy more memory.
- Allows speed gain when only a particular segment of the program is required for the execution of the program.
- This frees programmers from concern of memory limitations and helps implementing multiprogramming environment.

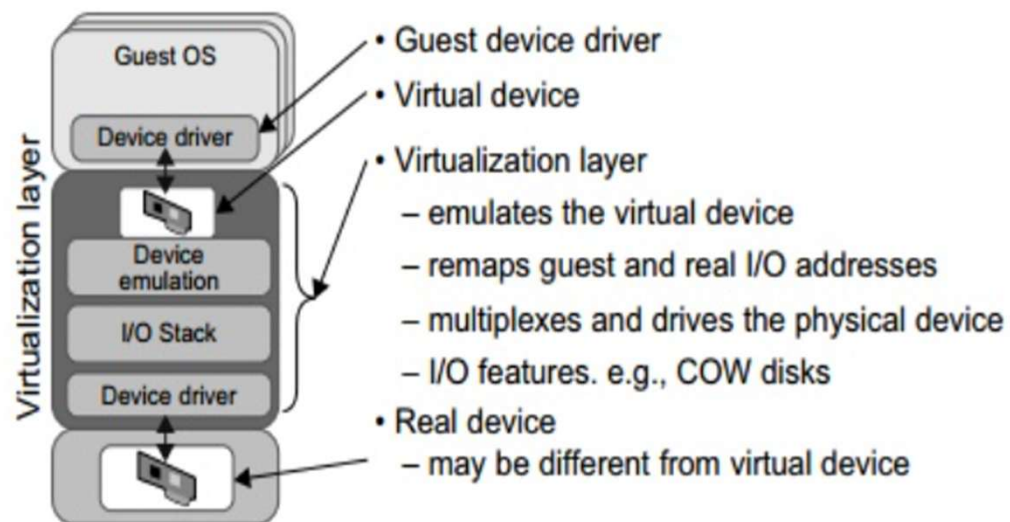
Disadvantages

- ❑ Applications may run slower.
- ❑ It takes more time to switch between applications.
- ❑ Less available hard drive space for the user's use.
- ❑ The possibility of Thrashing due to excessive Paging and Page faults

I/O VIRTUALIZATION

i/O virtualization involves managing the routing of I/O requests between virtual devices and the shared physical hardware. At the time of this writing, there are three ways to implement I/O virtualization: full device emulation, para-virtualization, and direct I/O. Full device emulation is the first approach for I/O virtualization.

Generally, this approach emulates well-known, real-world devices.



All the functions of a device or bus infrastructure, such as device enumeration, identification, interrupts, and DMA, are replicated in software. This software is located in the VMM and acts as a virtual device. The I/O access requests of the guest OS are trapped in the VMM which interacts with the I/O devices. The full device emulation approach is shown in Figure

A single hardware device can be shared by multiple VMs that run concurrently. However, software emulation runs much slower than the hardware it emulates

The para-virtualization method of I/O virtualization is typically used in Xen. It is also known as the split driver model consisting of a frontend driver and a backend driver. The frontend driver is running in Domain U and the backend driver is running in Domain 0. They interact with each other via a block of shared memory. The frontend driver manages the I/O requests of the guest OSes and the backend driver is responsible for managing the real I/O devices and multiplexing the I/O data of different VMs. Although para-I/O-virtualization achieves better device performance than full device emulation, it comes with a higher CPU overhead.

Direct I/O virtualization lets the VM access devices directly. It can achieve close-to-native performance without high CPU costs. However, current direct I/O virtualization implementations focus on networking for mainframes. There are a lot of challenges for commodity hardware devices.

For example, when a physical device is reclaimed (required by workload migration) for later reassign-ment, it may have been set to an arbitrary state (e.g., DMA to some arbitrary memory locations) that can function incorrectly or even crash the whole system. Since software-based I/O virtualization requires a very high overhead of device emulation, hardware-assisted I/O virtualization is critical. Intel VT-d supports the remapping of I/O DMA transfers and device-generated interrupts. The architecture of VT-d provides the flexibility to support multiple usage models that may run unmodified, special-purpose, or “virtualization-aware” guest OSes.

Another way to help I/O virtualization is via self-virtualized I/O (SV-IO)

The key idea of SV-IO is to harness the rich resources of a multicore processor.

All tasks associated with virtualizing an I/O device are encapsulated in SV-IO.

It provides virtual devices and an associated access API to VMs and a management API to the VMM. SV-IO defines one virtual interface (VIF) for every kind of virtualized I/O device, such as virtual network interfaces, virtual block devices (disk), virtual camera devices, and others.

The guest OS interacts with the VIFs via VIF device drivers. Each VIF consists of two message queues. One is for outgoing messages to the devices and the other is for incoming messages from the devices. In addition, each VIF has a unique ID for identifying it in SV-IO.

PARA VIRTUALIZATION

In computing, paravirtualization or para-virtualization is a **virtualization technique that presents a software interface to the virtual machines** which is similar, yet not identical to the underlying hardware–software interface

- The intent of the modified interface is to reduce the portion of the guest's execution time spent performing operations which are substantially more difficult to run in a virtual environment compared to a non-virtualized environment.
- The paravirtualization provides specially defined 'hooks' to allow the guest(s) and host to request and acknowledge these tasks, which would otherwise be executed in the virtual domain (where execution performance is worse).
- A successful paravirtualized platform may allow the [virtual machine monitor](#) (VMM) to be simpler (by relocating execution of critical tasks from the virtual domain to the host domain), and/or reduce the overall performance degradation of machine execution inside the virtual guest.